

R pour jeunes économistes

RMarkdown et production de documents

J.-B. Guiffard

5 juin 2026

RMarkdown et production de documents

Objet de la séance

Cette dernière séance est consacrée à la production de documents avec **RMarkdown**.

L'objectif est de comprendre comment passer :

script R $\rightarrow$ document reproductible $\rightarrow$ rapport / slides / html / pdf

Nous verrons comment utiliser RMarkdown pour :

- rédiger du texte ;
- intégrer du code R ;
- produire automatiquement des tableaux ;
- intégrer des graphiques ;
- générer plusieurs formats de sortie ;
- automatiser la production de rapports.

$\Rightarrow$ RMarkdown permet de réunir dans un même fichier le raisonnement, le code et les résultats.

Pourquoi utiliser RMarkdown ?

Dans un travail de recherche, on produit souvent :

- des statistiques descriptives ;
- des graphiques ;
- des tableaux de régression ;
- des cartes ;
- des rapports intermédiaires ;
- des supports de présentation.

Avec un workflow classique, ces éléments sont souvent produits séparément puis copiés-collés dans Word, PowerPoint ou LaTeX.

Avec RMarkdown, on peut produire ces éléments directement à partir du code.

⇒ Cela réduit les erreurs manuelles et facilite la mise à jour des résultats.

RMarkdown comme outil de recherche reproductible

RMarkdown permet de produire un document à partir d'un fichier source unique.

Ce fichier contient :

- le texte ;
- le code ;
- les résultats ;
- les graphiques ;
- les tableaux ;
- les options du document.

texte + code + données → document final

Plan de la séance

La séance est organisée en six étapes.

- 1 Comprendre le principe de RMarkdown
- 2 Créer un premier document .Rmd
- 3 Écrire du texte avec la syntaxe Markdown
- 4 Insérer du code R et contrôler les chunks
- 5 Intégrer tableaux, graphiques et résultats
- 6 Automatiser et paramétrer des rapports

Comprendre RMarkdown

Qu'est-ce qu'un fichier RMarkdown ?

Un fichier RMarkdown est un fichier texte avec l'extension `.Rmd`.

Il permet de mélanger :

- du texte ;
- des titres ;
- des équations ;
- du code R ;
- des sorties produites par R.

Exemple :

```
mean(c(1, 2, 3, 4, 5))
```

Après compilation, le résultat est intégré dans le document final.

Le principe général

RMarkdown fonctionne en deux étapes.

`.Rmd` → `.md` → document final

Première étape :

- le package `knitr` exécute le code R ;
- les résultats sont insérés dans un document Markdown.

Deuxième étape :

- `pandoc` transforme le document en HTML, PDF, Word ou slides.

Quels formats peut-on produire ?

À partir d'un même fichier `.Rmd`, on peut produire plusieurs formats.

- `html_document` : rapport HTML ;
- `pdf_document` : rapport PDF ;
- `word_document` : document Word ;
- `beamer_presentation` : slides PDF ;
- `ioslides_presentation` : slides HTML.

Le contenu reste le même, mais le format de sortie peut changer.

Pourquoi c'est utile pour un doctorant ?

RMarkdown est particulièrement utile pour :

- produire des rapports intermédiaires ;
- documenter une analyse empirique ;
- préparer un support de séminaire ;
- partager des résultats avec des co-auteurs ;
- produire des annexes reproductibles ;
- automatiser des tableaux ou graphiques.

Exemple : Lorsque les données changent, il suffit de recompiler le document pour mettre à jour les résultats.

Créer un premier document RMarkdown

Structure générale d'un document

Un document RMarkdown contient deux grandes parties :

- 1 un en-tête YAML ;
- 2 un corps de document.

L'en-tête définit les métadonnées et le format de sortie.

Le corps contient le texte, le code et les résultats.

title: "Analyse des émissions de CO2" author: "Prénom Nom" date:
"2026-06-05" output: html_document

Exemple d'en-tête simple

title: "Analyse des émissions de CO2" subtitle: "Exemple de rapport reproductible" author: "Jean-Baptiste Guiffard" date: "5 juin 2026" output: html_document

Cet en-tête indique :

- le titre du document ;
- le sous-titre ;
- l'auteur ;
- la date ;
- le format de sortie.

Plusieurs formats de sortie

On peut demander plusieurs formats dans le même fichier.

```
title: "Analyse des émissions de CO2" author: "Prénom Nom" output:  
html_document: toc: true pdf_document: toc: true word_document:  
default
```

Cela permet de produire :

- un rapport HTML ;
- un rapport PDF ;
- un document Word.

Le chunk setup

Au début du document, on ajoute souvent un chunk appelé setup.

```
knitr::opts_chunk$set( echo = TRUE, warning = FALSE, message = FALSE )
```

Ce chunk permet de définir les options par défaut.

Ici :

- le code est affiché ;
- les warnings ne sont pas affichés ;
- les messages ne sont pas affichés.

Écrire du texte avec Markdown

Titres, texte et listes

La syntaxe Markdown est légère.

Titre de niveau 1

Titre de niveau 2

Titre de niveau 3

Ceci est du texte avec *de l'italique* et **du gras**.

- premier élément
- deuxième élément
- troisième élément

Markdown permet d'écrire un document lisible même avant compilation.

Liens, images et équations

On peut insérer un lien :

```
[Our World in Data](https://ourworldindata.org/)
```

On peut insérer une image :

```
![Titre de la figure](figures/ma_figure.png){width=70%}
```

On peut écrire une équation :

```
$$  
Y_i = \alpha + \beta X_i + \varepsilon_i  
$$
```

Insérer un résultat dans le texte

On peut insérer un résultat R directement dans une phrase.

La base contient `nrow(co2_analysis)` observations.

Après compilation, R remplace le code par le résultat.

Exemple :

La base contient 4231 observations.

Le code inline évite de recopier manuellement des chiffres dans le texte.

Insérer du code R

Qu'est-ce qu'un chunk ?

Un chunk est un bloc de code R inséré dans le document.

```
{r}
```

Lors de la compilation :

- le code peut être exécuté ;
- le résultat peut être affiché ;
- les graphiques peuvent être intégrés.

Charger les packages

Dans un rapport reproductible, les packages doivent être chargés dans le document.

```
{r packages, message=FALSE, warning=FALSE}
```

Cela permet au document de fonctionner dans une nouvelle session R.

Importer les données

Les données doivent aussi être importées dans le document.

```
{r import-data, eval=FALSE} co2_raw <-  
read_csv("data/raw/owid-co2-data.csv")
```

On utilise de préférence des chemins relatifs.

À éviter :

```
{r, eval=FALSE}  
read_csv("C:/Users/Jean-Baptiste/Desktop/owid-co2-data.csv")
```


Construire une base d'analyse

On peut construire la base directement dans le document.

```
{r build-data, eval=FALSE} co2_analysis <- co2_raw %>% filter( year >= 1990,  
  !is.na(iso_code), !is.na(co2_per_capita), !is.na(population), !is.na(gdp),  
  population > 0, gdp > 0, co2_per_capita > 0 ) %>% mutate( gdp_per_capita  
  = gdp / population, log_gdp_pc = log(gdp_per_capita), log_co2_pc =  
  log(co2_per_capita) )
```

Contrôler les chunks

Les principales options

Les chunks peuvent être contrôlés avec des options.

- `echo` : afficher ou non le code ;
- `eval` : exécuter ou non le code ;
- `include` : inclure ou non le code et les résultats ;
- `warning` : afficher ou non les warnings ;
- `message` : afficher ou non les messages ;
- `fig.width` et `fig.height` : taille des figures.

Exemple : afficher le résultat mais pas le code

echo=FALSE

```
{r desc-table, eval=FALSE} co2_analysis %>% summarise( observations = n(),  
countries = n_distinct(country), mean_co2_pc = mean(co2_per_capita,  
na.rm = TRUE) )
```

- le code est exécuté ;
- le résultat est affiché ;
- le code n'apparaît pas dans le document.

Exemple : montrer le code sans l'exécuter

```
{r exemple-code, eval=FALSE} co2_analysis %>% group_by(year) %>%  
summarise(mean_co2 = mean(co2_per_capita, na.rm = TRUE))
```

- le code est affiché ;
- le code n'est pas exécuté ;
- aucun résultat n'est produit.

Tableaux dans RMarkdown

Produire une table descriptive

On peut produire une table à partir du code R.

```
{r, eval=FALSE}
```

```
desc_table <- co2_analysis %>% summarise( Observations = n(), Pays =  
n_distinct(country), CO2 moyen par habitant = mean(co2_per_capita,  
na.rm = TRUE), PIB moyen par habitant = mean(gdp_per_capita, na.rm  
= TRUE) )
```

```
knitr::kable(desc_table, digits = 2)
```

Les chiffres du tableau sont recalculés automatiquement à chaque compilation.

Table par groupe

On peut produire des tableaux par année ou par pays.

```
{r, eval=FALSE}
```

```
year_stats <- co2_analysis %>% group_by(year) %>% summarise(  
  mean_co2_pc = mean(co2_per_capita, na.rm = TRUE), median_co2_pc =  
  median(co2_per_capita, na.rm = TRUE), n_countries = n_distinct(country),  
  .groups = "drop" )  
  
knitr::kable(head(year_stats, 10), digits = 2)
```


Tableaux plus avancés

Pour des besoins plus avancés, on peut utiliser plusieurs packages.

- `knitr::kable()` : tableaux simples ;
- `kableExtra` : mise en forme avancée ;
- `gt` : tableaux modernes ;
- `flextable` : tableaux pour Word et PowerPoint ;
- `modelsummary` : tableaux de régressions et statistiques descriptives.

Conseil

Commencer simple avec `kable()`, puis utiliser des packages plus avancés selon les besoins.

Graphiques dans RMarkdown

Intégrer un graphique ggplot

Les graphiques produits par R sont intégrés automatiquement.

```
{r graph-distribution, fig.width=7, fig.height=4}
```

```
{r, eval=FALSE} ggplot(co2_analysis, aes(x = log_co2_pc)) +  
  geom_histogram(bins = 40) + theme_minimal() + labs( title = "Distribution  
du log des émissions de CO2 par habitant", x = "log(CO2 par habitant)", y =  
"Nombre d'observations" )
```

Graphique temporel

```
{r graph-time, fig.width=7, fig.height=4} {r, eval=FALSE}

co2_year <- co2_analysis %>% group_by(year) %>% summarise(
  mean_co2_pc = mean(co2_per_capita, na.rm = TRUE), .groups = "drop" )

ggplot(co2_year, aes(x = year, y = mean_co2_pc)) + geom_line() +
  theme_minimal() + labs( title = "Évolution moyenne des émissions de CO2
  par habitant", x = "Année", y = "CO2 par habitant moyen" )
```

Nommer les chunks facilite la résolution des erreurs.

Régressions et résultats

Une première régression

On peut estimer un modèle directement dans le document.

```
{r, eval=FALSE} model_1 <- lm(log_co2_pc ~ log_gdp_pc, data =  
co2_analysis)
```

```
summary(model_1)
```

Cette régression décrit une relation entre PIB par habitant et émissions de CO2 par habitant.

Présenter une table de régression

```
{r regression-table, echo=FALSE}
```

```
{r, eval=FALSE}
```

```
modelsummary::modelsummary( model_1, stars = TRUE, title = "Relation  
entre PIB par habitant et émissions de CO2" )
```

Le tableau est produit automatiquement à partir de l'objet `model_1`.

Rapports automatisés

Pourquoi automatiser un rapport ?

Un rapport automatisé est utile lorsqu'un document doit être mis à jour régulièrement.

Exemples :

- rapport mensuel ;
- tableau de bord ;
- note de suivi ;
- rapport par pays ;
- rapport par région ;
- rapport par année ;
- annexe empirique.

Le rapport est produit à partir du code : si les données changent, le document peut être mis à jour automatiquement.

Compiler avec render()

On peut compiler un fichier RMarkdown depuis R.

```
{r, eval=FALSE} rmarkdown::render("rapport_co2.Rmd")
```

On peut aussi choisir un environnement propre.

```
{r, eval=FALSE} rmarkdown::render( input = "rapport_co2.Rmd", envir =  
new.env() )
```

Rapports paramétrés

Un rapport paramétré permet de produire plusieurs documents à partir du même modèle.

Exemple : produire un rapport pour un pays donné.

Dans le YAML :

```
title: "Rapport CO2" output: html_document params: country_name:
"France"
```

Dans le code :

```
{r, eval=FALSE} co2_country <- co2_analysis %>% filter(country ==
params$country_name)
```

Générer un rapport pour un pays

```
{r, eval=FALSE} rmarkdown::render( input = "rapport_co2_parametre.Rmd",  
  params = list(country_name = "Brazil"), output_file =  
  "rapport_co2_Brazil.html", envir = new.env() )
```

On peut produire le même rapport pour un autre pays :

```
{r, eval=FALSE} rmarkdown::render( input = "rapport_co2_parametre.Rmd",  
  params = list(country_name = "India"), output_file =  
  "rapport_co2_India.html", envir = new.env() )
```

Générer plusieurs rapports

```
{r, eval=FALSE} countries <- c("France", "Brazil", "India")  
  
for (pays in countries) { rmarkdown::render( input =  
"rapport_co2_parametre.Rmd", params = list(country_name = pays),  
output_file = paste0("rapport_co2_", pays, ".html"), envir = new.env() ) }
```

Usage

Très utile pour produire automatiquement des rapports par pays, région, secteur ou année.

Bonnes pratiques

Organiser son projet

Un projet RMarkdown doit être organisé clairement.

```
projet_rmarkdown/  
├── data/  
│   ├── raw/  
│   └── clean/  
├── scripts/  
├── reports/  
├── outputs/  
│   ├── figures/  
│   └── tables/  
└── projet_rmarkdown.Rproj
```

Les erreurs classiques

Quelques erreurs reviennent souvent avec RMarkdown.

- Utiliser des chemins absolus.
- Oublier de charger un package dans le document.
- Utiliser un objet présent dans l'environnement mais non créé dans le `'Rmd'`.
- Ne pas nommer les chunks.
- Copier-coller manuellement des chiffres dans le texte.
- Masquer les messages d'erreur au lieu de les comprendre.
- Compiler rarement le document.
- Ne pas vérifier que le document compile dans une nouvelle session.

Conclusion

Merci beaucoup pour votre attention !

Dernière discussion :

Comment pourriez-vous utiliser RMarkdown dans votre propre thèse ?